Baseline Notebook

Setup Environment

In [111…
```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

Student Information

In [112…
```
group_name = ""
student_name = "Yi An Tsai"
student_id = "25532767"
```

In [113…
```
# Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

group_name

In [114…
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Yi An Tsai

In [115…
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

25532767

---

0. Python Packages

0.a Install Additional Packages

> If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

0.b Import Packages

In [117…
```
import pandas as pd
import altair as alt
```

A. Assess Baseline Model

In [118…
```
# DO NOT MODIFY THE CODE IN THIS CELL
# Load data
try:
  X_train = pd.read_csv(at.folder_path / 'X_train.csv')

  X_val = pd.read_csv(at.folder_path / 'X_val.csv')

  X_test = pd.read_csv(at.folder_path / 'X_test.csv')

except Exception as e:
  print(e)
```

A.1 Generate Predictions with Baseline Model

In [119…
```
from sklearn.dummy import DummyClassifier
```

In [120…
```
# have a look
X_train.head()
```

Out[120…

	sales_order_id	sales_order_detail_id	order_quantity	special_offer_id	unit_price	unit_price_discount	line_total	category_name	online_order_flag	cust
0	0fbc9d71-4176-4c77-92c6-d7ede2a9ce33	354.0	1.0	a1444fcd-4b51-4227-94cf-b13c1a2a6281	3399.99	0.0	3399.99	Bikes	1.0	5 27 4272
1	18609f9d-aedc-4665-bbc8-5d7a99599915	357.0	1.0	a1444fcd-4b51-4227-94cf-b13c1a2a6281	3399.99	0.0	3399.99	Bikes	1.0	7 fa 9e413
2	0570389a-6bfa-45c2-85a4-36784fa7b5b1	359.0	1.0	a1444fcd-4b51-4227-94cf-b13c1a2a6281	3578.27	0.0	3578.27	Bikes	1.0	7 9f 6828
3	b48cdf2f-270a-41d0-9cde-36616fecd302	360.0	1.0	a1444fcd-4b51-4227-94cf-b13c1a2a6281	3374.99	0.0	3374.99	Bikes	1.0	c 64 4322
4	03533644-6a12-499b-9726-aaa1088c56a5	361.0	1.0	a1444fcd-4b51-4227-94cf-b13c1a2a6281	3399.99	0.0	3399.99	Bikes	1.0	d 77 0f976

In [121…
```
# at customer level
X_train = X_train.groupby('customer_id').first().reset_index()
X_val = X_val.groupby('customer_id').first().reset_index()
X_test = X_test.groupby('customer_id').first().reset_index()
```

In [122…
```
#drop unused columns
drop_list = [
    'sales_order_id',
    'sales_order_detail_id',
    'order_quantity', 'unit_price',
    'unit_price_discount',
    'line_total',
    'customer_id',
    'order_date',
    'sub_total',
    'tax_amount',
    'freight'
    ]

X_train = X_train.drop(drop_list, axis=1)
X_val = X_val.drop(drop_list, axis=1)
X_test = X_test.drop(drop_list, axis=1)
```

In [123…
```
X_train.head()
```

Out[123…

	special_offer_id	category_name	online_order_flag	total_due	sales_country	sales_group	customer_country	customer_group
0	a1444fcd-4b51-4227-94cf-b13c1a2a6281	Accessories	1.0	110.4337	Germany	Europe	Germany	Europe
1	a1444fcd-4b51-4227-94cf-b13c1a2a6281	Bikes	1.0	2673.0613	United Kingdom	Europe	United Kingdom	Europe
2	a1444fcd-4b51-4227-94cf-b13c1a2a6281	Bikes	1.0	1105.4834	Australia	Pacific	Australia	Pacific
3	a1444fcd-4b51-4227-94cf-b13c1a2a6281	Bikes	1.0	2288.9187	Australia	Pacific	Australia	Pacific
4	a1444fcd-4b51-4227-94cf-b13c1a2a6281	Accessories	1.0	49.7029	France	Europe	France	Europe

In [124…
```
from sklearn.preprocessing import StandardScaler


# encoding on cat_col
categorical_features = ['special_offer_id', 'sales_country', 'sales_group', 'customer_country', 'customer_group']
X_train = pd.get_dummies(X_train, columns=categorical_features, drop_first=False)

# columns list after encoding
train_columns = X_train.columns

# apply on val set
X_val = pd.get_dummies(X_val, columns=categorical_features, drop_first=False)
X_val = X_val.reindex(columns=train_columns, fill_value=0)

# repeat on test set
X_test = pd.get_dummies(X_test, columns=categorical_features, drop_first=False)
X_test = X_test.reindex(columns=train_columns, fill_value=0)
```

```
#  scaling on num_col
scaler = StandardScaler()
num_col = ['total_due']

# fit on X_train and transform all datasets
X_train[num_col] = scaler.fit_transform(X_train[num_col])
X_val[num_col] = scaler.transform(X_val[num_col])
X_test[num_col] = scaler.transform(X_test[num_col])
```

In [141…
```
X_val.head()
```

Out[141…

	online_order_flag	total_due	special_offer_id_6468cb43-e8b4-475f-b705-f4d37f1c59eb	special_offer_id_a09a7508-0e3b-42aa-942e-dde8428b1f62	special_offer_id_a1444fcd-4b51-4227-94cf-b13c1a2a6281	special_offer_id_acc417ee-ae0e-4d78-8f48-3f541565c54e	special_offer_id f166 c
0	1.0	-0.321688	0	0	True	0	
1	1.0	-0.325021	0	0	True	0	
2	1.0	-0.115357	0	0	True	0	
3	1.0	0.089361	0	0	True	0	
4	1.0	-0.321688	0	0	True	0	

In [125…
```
# Separate target variable
y_train = X_train['category_name']
y_val = X_val['category_name']
y_test = X_test['category_name']

X_train = X_train.drop(columns=['category_name'])
X_val = X_val.drop(columns=['category_name'])
X_test = X_test.drop(columns=['category_name'])
```

In [140…
```
X_train.head()
```

Out[140…

	online_order_flag	total_due	special_offer_id_6468cb43-e8b4-475f-b705-f4d37f1c59eb	special_offer_id_a09a7508-0e3b-42aa-942e-dde8428b1f62	special_offer_id_a1444fcd-4b51-4227-94cf-b13c1a2a6281	special_offer_id_acc417ee-ae0e-4d78-8f48-3f541565c54e	special_offer_id f166 c
0	1.0	-0.310878	False	False	True	False	
1	1.0	0.107221	False	False	True	False	
2	1.0	-0.148533	False	False	True	False	
3	1.0	0.044548	False	False	True	False	
4	1.0	-0.320787	False	False	True	False	

In [155…
```
# verify that X and y have matching row counts
print(X_train.shape)
print(y_train.shape)
```

```
(6674, 20)
(6674,)
```

In [128…
```
# train and fit
dummy = DummyClassifier(strategy='most_frequent', random_state=42)
dummy.fit(X_train, y_train)

# predict
y_pred = dummy.predict(X_train)
y_val_pred = dummy.predict(X_val)
```

A.2 Selection of Performance Metrics

> Provide some explanations on why you believe the performance metrics you chose is appropriate

In [129…
```
from sklearn.metrics import accuracy_score, classification_report, f1_score
```

In [130…
```
performance_metrics_explanations = """
The confusion matrix is selected as it provides comprehensive visual information about the model's performance across all categories,
showing which classes are correctly predicted and which are confused with each other.

Accuracy and F1-score are also used as evaluation metrics.
Accuracy measures overall prediction correctness,
while F1-score is more appropriate for multi-class classification
as it accounts for class imbalance by evaluating each class independently before averaging (macro average).
"""
```

In [131…
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='performance_metrics_explanations', value=performance_metrics_explanations)
```

performance_metrics_explanations

The confusion matrix is selected as it provides comprehensive visual information about the model's performance across all categories, showing which classes are correctly predicted and which are confused with each other. Accuracy and F1-score are also used as evaluation metrics. Accuracy measures overall prediction correctness, while F1-score is more appropriate for multi-class classification as it accounts for class imbalance by evaluating each class independently before averaging (macro average).

A.3 Baseline Model Performance

> Provide some explanations on model performance

In [152…
```
# Accuracy and F1-score of two sets
print(f"Train set:   Accuracy: {accuracy_score(y_train, y_pred):.4f}")
print(f"Train set:   F1 score: {f1_score(y_train, y_pred, average='macro')  :.4f}")
print(f"Val set   Accuracy: {accuracy_score(y_val, y_val_pred):.4f}")
print( f"Val set   F1 score: {f1_score(y_val, y_val_pred, average='macro')  :.4f}")
```

```
Train set:   Accuracy: 0.6310
Train set:   F1 score: 0.1934
Val set   Accuracy: 0.5532
Val set   F1 score: 0.1781
```

In [153…
```
#  train set metrics
report = classification_report(y_train, y_pred, output_dict=True)
pd.DataFrame(report).transpose().round(2)
```

Out[153…

	precision	recall	f1-score	support
Accessories	0.00	0.00	0.00	2076.00
Bikes	0.63	1.00	0.77	4211.00
Clothing	0.00	0.00	0.00	318.00
Components	0.00	0.00	0.00	69.00
accuracy	0.63	0.63	0.63	0.63
macro avg	0.16	0.25	0.19	6674.00
weighted avg	0.40	0.63	0.49	6674.00

In [154…
```
# val set metrics
report = classification_report(y_val, y_val_pred, output_dict=True)
pd.DataFrame(report).transpose().round(2)
```

Out[154…

	precision	recall	f1-score	support
Accessories	0.00	0.00	0.00	1070.00
Bikes	0.55	1.00	0.71	1591.00
Clothing	0.00	0.00	0.00	189.00
Components	0.00	0.00	0.00	26.00
accuracy	0.55	0.55	0.55	0.55
macro avg	0.14	0.25	0.18	2876.00
weighted avg	0.31	0.55	0.39	2876.00

In [137…
```
baseline_performance_explanations = """

The accuracy is 0.63 of train set and 0.55 of val set,
mostly contributed by the Bikes group, because this model has no hyperparameter tuning and is set to 'most_frequent' strategy,
which always predicts 'Bikes'.
Therefore, the recall for Bikes must be 1.0 since all predictions from this model are 'Bikes',
and accuracy of val set is 0.55 due to the imbalanced distribution of this dataset.
F1-score shows an extremely low value due to zero precision and recall for all non-Bikes categories.

"""
```

In [138…
```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='baseline_performance_explanations', value=baseline_performance_explanations)
```

baseline_performance_explanations

The accuracy of the train and val sets are around 0.91~0.94, while the F1-scores are approximately 0.64~0.58. From the confusion matrics report, we can know that the performance of clothing of both sets are lower than what we expected, even lower than the minimum category- compoenets (0.34 f1 score), with merely 0.02 f1 score in the val set. It points out that the class imbalance in the target variable causes the model to perform poorly on the 'Clothing' classes, instead of three other classes. As for the feature importance, the total due accounts for a significant level over other features, with around 5.5 importance. Although online flag has over 70% category belongs to 1, it also show a key value.